

Faculty of Engineering & Technology
Electrical & Computer Engineering Department

Advanced Computer Architecture ENCS5331

Research Project

FPGA-Based Hardware Accelerator for Real-Time Edge Detection in Image Processing

---

Prepared by:

Qasim Batrawi 1220204

Taleed Hamadneh 1220006

Instructor: Prof. Ayman Hroub

Date: Jan 16, 2026

Abstract

Edge detection is a fundamental operation in image processing, widely used in real-time applications such as computer vision, robotics, and video surveillance. Software-based implementations of edge detection algorithms, particularly the Sobel operator, suffer from high latency and power consumption due to sequential pixel processing on general-purpose processors. Although many FPGA-based implementations have been proposed, several prior works either rely on frame-based processing, fixed image sizes, or do not clearly demonstrate scalable throughput improvements under a consistent evaluation setup. This report presents the design and implementation of an FPGA-based hardware accelerator for real-time edge detection using a fully pipelined Sobel architecture. The proposed solution employs a two-wide parallel datapath, allowing two pixels to be processed per clock cycle to significantly increase throughput. The accelerator consists of several pipeline stages, including grayscale conversion, 3×3 sliding window generation, Sobel convolution, absolute value computation, gradient magnitude approximation, and thresholding. To reduce hardware complexity, multiplier-free shift-and-add arithmetic and a magnitude approximation based on $|G_x| + |G_y|$ are used. The hardware design is implemented in SystemVerilog, while Python is utilized for software pre-processing and post-processing, including image reconstruction and validation. The hardware-generated edge-detected images are compared against a software-based Sobel implementation using visual inspection as well as quantitative metrics such as Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR). Experimental results on two 256×256 test images show MSE values of 19384.87 and 17566.67, corresponding to PSNR values of 5.26 dB and 5.68 dB, respectively, which are better than or comparable to the results reported in several existing FPGA-based edge detection implementations. In addition, the measured simulation runtime improves from 655575 ns (one-wide) to 327905 ns (two-wide), achieving an experimental speed-up of approximately $2.00\times$, which confirms near-ideal throughput scaling for the two-wide architecture.

Contents

1 Introduction **1**

2 Background **2**

3 Related Work **3**

4 Proposed Solution Description **11**
4.1 System-Level Architecture (General Block Diagram) 11
4.2 Detailed Two-Wide Datapath Overview 11
4.3 Stage-Level Detailed Block Diagrams 12

5 Implementation and Experimental Setup **13**
5.1 Software Implementation Pre-processing 13
5.2 Hardware Architecture 13
5.2.1 Dual-Pixel Read and Grayscale Conversion 13
5.2.2 3×3 Sliding Window Generation Stage 14
5.2.3 Convolution Stage 14
5.2.4 Absolute and Gradient Magnitude Calculation Stages 15
5.2.5 Edge Thresholding Stage 15
5.3 Software Implementation Post-processing 15

6 Experimental Results and Discussion **16**
6.1 Qualitative Results (Visual Verification) 16
6.2 Quantitative Accuracy (MSE/PSNR) 17
6.3 Timing, Throughput, and Speed-up Analysis 18
6.3.1 Pipeline Timing Model (k=4 stages) 18
6.3.2 Throughput Scaling by Two-Way Parallelism (1-wide vs 2-wide) . 18
6.4 Comparison with Prior Work 19

7 Conclusion and Future Work **22**

List of Figures

1 General block diagram of the proposed accelerator. 11
2 Detailed two-wide Sobel datapath overview. 11
3 Stage-level block diagrams of the proposed accelerator. 12
4 Qualitative comparison for Test image 1: original image and Sobel edge maps from the Python reference and the RTL pipeline. 16
5 Qualitative comparison for Test image 2: original image and Sobel edge maps from the Python reference and the RTL pipeline. 17
6 Measured simulation runtime for processing one full frame using one-wide and two-wide architectures. 19
7 Measured speed-up as a function of architecture width, showing near-ideal 2× scaling for the two-wide design. 19
8 PSNR comparison between the proposed design and selected reference implementations (average values used for the proposed method). 20
9 MSE comparison between the proposed design and selected reference implementations (average values used for the proposed method). 20
10 Bar chart comparison of normalized throughput in pixels per cycle. . . . 21

List of Tables

1 Summary of Related FPGA-Based Edge Detection Studies 8
2 Accuracy of RTL output compared to Python reference. 18
3 Measured throughput improvement from two-way parallelism (same pipeline depth $k = 4$). 19
4 PSNR and MSE comparison with selected reference implementations. . . 20
5 Normalized throughput comparison using pixels-per-cycle metric. 21

1 Introduction

In many real-time image processing applications, extracting meaningful structural information from visual data is a critical requirement, and edge detection plays a central role in achieving this goal by highlighting intensity changes that correspond to object boundaries and structural features. Such processing is widely used in fields including video surveillance, robotics, and autonomous systems. Among the various edge detection techniques, the Sobel operator is commonly employed due to its simplicity and effectiveness in detecting horizontal and vertical edges. However, when edge detection algorithms are implemented on general-purpose processors, they become computationally demanding, as software-based solutions rely on sequential pixel-by-pixel convolution operations that introduce high latency and limit real-time performance, particularly for high-resolution images. Moreover, meeting real-time constraints on CPUs often results in increased power consumption, which is undesirable in embedded and real-time environments. To address these challenges, numerous studies have investigated FPGA-based hardware acceleration for edge detection, demonstrating that pipelined and parallel FPGA architectures can significantly improve throughput by exploiting customized data paths and fine-grained parallelism. Despite these advancements, designing efficient accelerators that balance performance, hardware complexity, and scalability remains an open research area. In this project, an FPGA-based hardware accelerator for real-time edge detection is proposed, implementing a fully pipelined Sobel architecture that includes window generation, convolution, absolute value computation, gradient magnitude calculation, and thresholding. Additionally, the design increases throughput by processing two image streams in parallel, where the input image is divided into two regions, each handled by an independent Sobel pipeline, resulting in a modular and high-throughput architecture suitable for real-time image processing applications.

2 Background

Edge detection is a fundamental operation in image processing and computer vision, as it identifies significant structural information by locating edges, which are regions in an image where brightness or color changes abruptly. These changes typically correspond to object boundaries, making edge detection a critical preprocessing step for tasks such as object recognition, segmentation, tracking, and scene understanding. Mathematically, edges are detected by measuring the rate of change of pixel intensity, known as the image gradient, which is commonly computed using convolution-based operators. Classical edge detection algorithms, including Roberts, Prewitt, Sobel, Laplacian, and Canny, differ in complexity, noise sensitivity, and computational cost. Among these, the Sobel operator is widely used due to its balance between simplicity, noise suppression, and effectiveness, as it employs two small 3×3 convolution kernels to compute horizontal and vertical gradients simultaneously. However, when implemented on general-purpose processors (CPUs), edge detection suffers from high latency and power consumption because of sequential pixel-by-pixel processing and repeated convolution operations, which limits real-time performance for high-resolution images in applications such as autonomous systems, video surveillance, and robotics. Field-Programmable Gate Arrays (FPGAs) provide a compelling solution to these limitations by enabling hardware acceleration through parallelism, pipelining, and reconfigurability. Unlike CPUs and GPUs, FPGAs allow the implementation of custom data paths where multiple pixels and operations can be processed concurrently, often producing one output pixel per clock cycle after pipeline initialization. This streaming and parallel processing capability significantly reduces latency and improves energy efficiency, making FPGAs well suited for real-time image processing in resource-constrained environments. Consequently, FPGA-based implementations of edge detection algorithms have gained increasing attention, as they offer high throughput, low power consumption, and architectural flexibility while maintaining acceptable edge detection accuracy. These advantages motivate the development of FPGA-based hardware accelerators for real-time edge detection, addressing the performance and efficiency limitations of traditional software-based approaches.

3 Related Work

A Novel Sobel Edge Detection Accelerator Based on Reconfigurable Architecture.[1]

This paper proposes an FPGA-based Sobel edge detection accelerator designed specifically for real-time video processing. The authors implement a pixel-level, fine-grained parallel architecture that processes one pixel per clock cycle using a fully pipelined dataflow. A row buffer storage structure combined with a sliding 3×3 window enables continuous streaming convolution without storing entire frames in memory. By converting RGB video input to grayscale on-chip and applying Sobel convolution only during the active video region, the accelerator achieves real-time performance at resolutions up to 1024×768 at 60 fps. Experimental results show that the proposed design outperforms CPU-based software and similar FPGA accelerators, achieving approximately 10% higher performance while maintaining a high power-to-performance ratio, making it suitable for embedded real-time edge detection systems.

FPGA-Based Improved Sobel Operator Edge Detection.[2]

This paper proposes an FPGA-based real-time edge detection system using an improved Sobel operator designed to overcome the limited directional sensitivity of the traditional Sobel algorithm. Instead of detecting edges only in the horizontal and vertical directions, the authors extend the Sobel operator to eight directions (0°, 45°, 90°, 135°, 180°, 225°, 270°, and 315°), allowing richer and more accurate edge extraction. The system implements the entire image processing pipeline on FPGA, including grayscale conversion using the Y component of the YCbCr color space, median filtering for noise reduction, multi-directional Sobel convolution, and threshold-based edge detection. Leveraging FPGA parallelism, all eight convolution operations are performed simultaneously, enabling high-speed processing. Images captured by an OV5640 camera are processed in real time and displayed through a VGA interface. Experimental results demonstrate improved edge continuity, reduced noise, and more detailed edge information compared to the traditional Sobel operator, confirming the effectiveness of the proposed FPGA-based approach for real-time image processing applications.

FPGA implementation of sobel edge detection algorithm.[3]

This paper presents a modified Sobel edge detection algorithm tailored for FPGA implementation. The authors remove multipliers and square-root operations, replacing them with simple sign reversal and addition to reduce hardware complexity. The algorithm is first verified in MATLAB, achieving a high similarity index (SSIM = 96.43%) compared to the standard Sobel operator. For hardware realization, the design is implemented in Verilog on a Cyclone III FPGA, integrated with a camera interface (OV7670) and FIFO buffer for real-time processing. Using Quartus II and an open-source toolchain (Yosys, OpenVPR, Skywater PDK), the system achieves real-time edge detection while maintaining low power consumption. Although logic element usage increases by 22%, the method demonstrates an effective trade-off between computation simplicity and performance, serving as a pre-processor to accelerate subsequent image processing tasks. Limitations include a small test dataset, lack of detailed edge accuracy analysis, and

increased logic utilization, which may be a concern for very small FPGAs. This work is relevant to FPGA-based real-time edge detection projects, highlighting practical strategies for hardware-oriented algorithm optimization.

Fast Edge Detection Architecture using Different Levels of Parallelism on a FPGA.[4]

The paper presents a fast FPGA-based architecture for first-order derivative edge detection using Sobel, Prewitt, and Robert operators. To achieve high speed for real-time applications, the authors propose two parallel architectures: one-way and two-way parallelism. The one-way method processes 9 pixels at a time in a single image region, achieving a speed-up of 460×, while the two-way method splits the image into two regions and processes them simultaneously, doubling the speed-up to 920×. The design leverages pipelined modules, parallel horizontal and vertical convolutions, and simultaneous execution of multiple edge detectors, making it highly efficient on FPGA. Instead of calculating the true gradient magnitude (which requires square roots and multiple cycles), the hardware-friendly approximation G = Gx + Gy is used to maintain a single-clock-cycle throughput. Implementation was done in Verilog, simulated with ModelSim, and synthesized on a Cyclone IV GX FPGA. Experimental results confirm substantial acceleration over software-based edge detection. However, the design is limited by FPGA memory (small image sizes), simplified gradient computation (approximated magnitude), dependency on MATLAB for preprocessing/postprocessing, increasing resource usage with higher parallelism, and only supporting basic first-order edge operators. These weaknesses could be addressed by streaming large images, implementing approximate square root computation, moving preprocessing into hardware, optimizing resource usage, and supporting more advanced edge detection operators.

FPGA-Based Processor Acceleration for Image Processing Applications.[5]

This paper presents an FPGA-based hardware acceleration framework for image processing using a custom soft-core processor architecture called the Image Processing Processor (IPPro). Instead of designing fixed-function accelerators, the authors propose a programmable, dataflow-driven multicore FPGA architecture that exploits fine-grained pipelining, SIMD parallelism, and streaming data execution to accelerate computationally intensive image processing tasks. The system integrates multiple IPPro cores with a dataflow programming model (RVC-CAL), enabling efficient mapping of pixel-level and neighborhood-based operations commonly found in real-time vision applications. Experimental results on k-means clustering and traffic sign recognition demonstrate substantial speedups (up to 9.6×) and significantly higher power efficiency compared to CPU and GPU implementations. The work directly addresses the limitations of CPU-based real-time image processing by leveraging FPGA parallelism to achieve low-latency, energy-efficient acceleration, making it highly relevant to real-time edge detection and embedded vision systems.

Parallel Processing of Sobel Edge Detection on FPGA: Enhancing Real-Time Image Analysis.[6]

This paper presents an FPGA-based real-time edge detection system that implements an improved Sobel operator to address the limited directional sensitivity of the traditional Sobel algorithm. By extending edge detection from two directions to eight directions, the design captures richer edge information and improves edge continuity. The entire image processing pipeline—including grayscale conversion, median filtering for noise reduction, multi-directional Sobel convolution, and threshold-based edge extraction—is implemented directly on FPGA, exploiting hardware parallelism to perform all convolution operations simultaneously. Experimental results using images captured from an OV5640 camera and displayed via a VGA interface demonstrate real-time processing capability and improved edge quality compared to the conventional Sobel approach. The work highlights the effectiveness of FPGA-based parallel architectures for overcoming the latency and power limitations of CPU-based edge detection, making it relevant to real-time embedded image processing applications, although adaptability to varying illumination conditions remains limited due to the use of a fixed threshold.

FPGA Implementation of a Real-Time Edge Detection System Based on an Improved Canny Algorithm.[7]

In the improved Canny edge detection algorithm proposed by Laigong Guo and Sitong Wu, the gradient calculations complexity were significantly reduced. The Sobel gradient mask was used to calculate the gradient of the image, which was stored in a line buffer. Instead of calculating the gradient magnitude using square and square root, the gradient was approximated for each pixel by the sum of the absolute values of the horizontal and vertical gradients. To compute the gradient direction without division or arctangent operations, the 0°–180° range is divided into eight zones, each defined by tangent values approximated as powers of 2 for shift-based calculations. A Non maximum suppression (NMS) is then applied to keep the strongest pixel in the direction of the gradient. In order to calculate the threshold adaptively, a logarithmic operation is used in Otsu's algorithm instead of division operations. After the threshold calculation is completed, the value obtained is set as the high threshold value, and the low threshold is half fo it, and the edge is detected based in the value obtained from the NMS. Results showed that the proposed design achieves the best combination of processing time and hardware resource usage. Moreover, the resulted edge detected image have the same quality to those result with other papers. However, approximating results may reduce the accuracy, and the design only works for a fixed image size of 512×512.

A parallel Canny edge detection algorithm based on OpenCL acceleration.[8]

In this paper, the authors propose a CPU–GPU collaborative solution that splits the Canny algorithm into smaller tasks so that both CPU and GPU can work together efficiently. They define four main OpenCL kernel functions that are executed in sequence, with parallel work occurring inside each kernel to handle image processing operations such as filtering, gradient calculation, and suppression. To improve memory efficiency and throughput, the algorithm uses vectorized memory access, allowing multiple data

elements to be read and written at the same time. Additionally, the authors select an optimal number of work-items per work-group in each kernel to maximize resource utilization on the GPU. These strategies provide fine-grained parallelism and result in significant speed improvements over traditional CPU implementations. The results showed that the algorithm produces the same average gradients and standard deviations as other methods, while achieving faster processing times across different image sizes. However, the algorithm still uses a fixed threshold for edge detection, which can reduce the accuracy. Additionally, the optimal number of work-items was tested on a single GPU type, which may not be ideal for other GPUs.

An Efficient Approach for Edge Detection Hardware Accelerator for Real Time Video Segmentation using Laplacian Operator.[9]

The authors implemented a VLSI-based FPGA hardware accelerator for real-time video segmentation using the Laplacian edge detection operator, focusing on efficient memory management and control via a finite state machine. Their design achieves 44.4% computation efficiency by eliminating zeros in the 3×3 convolution kernel. Memory access is improved by handling four pixels at a time when writing or reading data, which reduces the number of memory operations and reduces bandwidth by four times. When reading pixels, a computation area storage method is used so that only three rows are needed for every four neighboring pixels, making the process faster and more efficient. This design achieved a memory bandwidth reduction of over 80%. However, it is limited to monochrome 720x480 images, and the paper does not provide quantitative results for processing time or power, only mentioning that the output quality is comparable to Sobel.

Implementation of Mainly Used Edge Detection Algorithms on FPGA.[10]

The paper compares the performance of Sobel, Prewitt, and Canny edge detection algorithms implemented on FPGA hardware versus MATLAB software. All three algorithms were implemented on the FPGA, and their results were compared to the MATLAB implementation. In MATLAB, the process was divided into three stages: pre-processing, edge detection, and post-processing. During pre-processing, the grayscale image was resized to 256×256 pixels and prepared as a 1D input for edge detection. In the edge detection stage, MATLAB blocks were used to apply the three algorithms, simulating the hardware implementation. In post-processing, the output data was converted back into an image and displayed. The results showed that the MATLAB implementation achieved a slightly lower MSE but a slightly higher PSNR compared to the FPGA hardware.

Efficient OpenCL Accelerators for Canny Edge Detection Algorithm on a CPU-FPGA Platform.[11]

The paper presents two implementations to accelerate the Canny edge detection algorithm on a CPU-FPGA platform using OpenCL. In the code-partitioned approach, the algorithm is split between the CPU and FPGA: the CPU performs smoothing and grayscale conversion and immediately sends processed image parts to the FPGA so that both units work concurrently. In the data-partitioned approach, the input image is divided into tiles using a Weighted Round Robin giving higher weight to the CPU because the FPGA is

approximately twice as fast so both CPU and FPGA execute the same kernel on different image sections in parallel. However, the design uses a fixed weight for both CPU and FPGA in round robin scheduling, which may not suit all systems, and it does not focus on optimizing the OpenCL kernel itself, where complex operations in Canny may slow-down the process. In results, the data-partitioned method achieved up to 4.8× speedup and a 55% average reduction in energy consumption compared to CPU-only, and 2.1× speedup compared to FPGA-only; the code-partitioned method also showed improved performance over CPU-only and 2.1× gain over FPGA-only for small image sizes.

Image Edge Detection in FPGA.[12]

In this paper, the authors use an FPGA to accelerate edge detection by converting the input image from RGB to grayscale to reduce computation and memory access, since processing grayscale data requires less memory than RGB. Since the RGB to grayscale conversion has division operations that are costly in hardware, they simplify the conversion by dividing by 2 and 4, which can be implemented with simple shift operations. Edge detection is then performed using a threshold comparison in Verilog, and the result is converted back to an image for display. However, the grayscale conversion equation they use is not accurate, which can introduce noise, and the use of a fixed threshold can reduce edge detection accuracy. The paper also does not show performance measures such as processing time or power consumption, and does not compare results to other hardware accelerators.

Summary Table

Table 1: Summary of Related FPGA-Based Edge Detection Studies

Ref.	The Solution	Contribution	Limitation	Alignment with the Proposed Work
[1]	FPGA-based reconfigurable Sobel edge detection accelerator using pixel-level fine-grained parallelism, row buffers, and sliding-window convolution for real-time video streams.	Fully pipelined pixel-level Sobel with row-buffer storage, enabling real-time video edge detection with improved performance and lower power than CPU solutions.	Higher design complexity due to synchronization, multiple line buffers, and handling video blanking intervals; optimized mainly for streaming video.	Uses the same streaming 3×3 concept, but with a modular RTL stage pipeline verified on static images; simpler to integrate and duplicate for parallel throughput.
[2]	Real-time FPGA edge detection using an improved 8-direction Sobel operator with parallel convolution, median filtering, and VGA display.	Extends Sobel from 2 to 8 directions and maps convolutions to parallel hardware, improving edge completeness and noise suppression.	Fixed global threshold limits robustness under varying illumination; limited quantitative analysis of resources and power.	Thresholding is a separate parameterized RTL module (easy to tune), and we add quantitative evaluation (MSE = 17566.6719, PSNR = 5.6839 dB).
[3]	FPGA-based Sobel edge detection using a modified algorithm removing multipliers and square-root operations, implemented on Cyclone III with camera and FIFO.	Reduces hardware complexity while maintaining high similarity to standard Sobel output; suitable as a real-time pre-processing stage.	Increased logic usage and limited evaluation dataset; limited analysis of edge localization and noise sensitivity.	We strengthen evaluation by reporting quantitative metrics (MSE/PSNR) and validating full-frame reconstruction consistently.
[4]	Fast FPGA pipelined architecture supporting one-way, two-way, and m-way parallelism for multiple operators.	Processes multiple pixels per cycle, supports Sobel/Prewitt/Roberts, large speedups, and reduced pipeline stalls.	Scalability constrained by FPGA memory and resource growth; approximate magnitude; MATLAB preprocessing dependence.	We use controlled two-way parallelism (two pipelines) to scale throughput with bounded resources.

Continued on next page

Table 1 – continued from previous page

Ref.	The Solution	Contribution	Limitation	Alignment with the Proposed Work
[5]	Programmable FPGA image processing accelerator using a multicore soft processor (IPPro) with dataflow execution and SIMD streaming.	Reconfigurable platform for image processing with speedup and improved energy efficiency versus CPU/GPU for vision workloads.	High design complexity; not specialized for Sobel kernels; performance may be limited by data movement.	We avoid soft processors/toolchains by implementing a fixed-function Sobel RTL datapath, scalable via straightforward pipeline duplication.
[6]	Real-time FPGA Sobel accelerator using an improved 8-direction Sobel algorithm with modular parallel processing.	Improved Sobel (2→8 directions) using fixed-point arithmetic and shift operations, achieving real-time performance at high resolutions with reduced multiplier usage.	Focuses mainly on throughput and does not provide a strong quantitative comparison of output accuracy across different images and conditions.	We keep standard Sobel for clarity, use parameterized thresholding, and report quantitative results (MSE/PSNR).
[7]	FPGA-based edge detection using an improved Canny design with approximations and adaptive thresholding.	Reduces divisions and square-root operations; uses approximate magnitude and direction; proposes faster adaptive threshold computation.	Approximation may reduce precision; sequential image processing reduces throughput; evaluated on limited image sizes.	The proposed work targets higher throughput with a fully pipelined Sobel datapath and parallel processing, trading advanced Canny features for deterministic timing and simpler hardware.
[8]	Parallel Canny edge detection using OpenCL with CPU–GPU cooperation and vectorized memory access.	Demonstrates heterogeneous acceleration and compares OpenCL with CUDA and OpenMP approaches across image sizes.	Fixed threshold reduces robustness; tuning depends on specific GPU; limited power comparison.	Parallelism is implemented directly in RTL (two pipelines), avoiding GPU/runtime tuning; thresholding remains parameterized for practical tuning.

Continued on next page

Table 1 – continued from previous page

Ref.	The Solution	Contribution	Limitation	Alignment with the Proposed Work
[9]	VLSI-based FPGA accelerator for video segmentation using the Laplacian edge detection operator.	Reduces memory bandwidth via optimized memory access and FSM control; reports large bandwidth reduction.	It is tested on one fixed video size and mostly reports memory savings, not detailed accuracy results.	Our design is a clear modular Sobel pipeline, and we evaluate the output accuracy in a consistent way for comparison.
[10]	MATLAB/Simulink and FPGA implementation of Sobel, Prewitt, and Canny using Xilinx System Generator.	Rapid FPGA prototyping using XSG blocks; compares software and FPGA results across algorithms.	It does not clearly measure core hardware metrics such as pipeline latency, throughput (pixels per cycle), or scalability beyond the fixed setup.	Our design is an explicit pipelined RTL datapath with a clear valid-based flow, so latency and throughput are directly defined by the pipeline stages, and the architecture scales by duplicating the pipeline (two-wide).
[11]	CPU–FPGA heterogeneous system accelerating Canny via code and data partitioning.	Improves overall system performance and energy using cooperative CPU and FPGA execution.	Fixed workload scheduling; OpenCL kernel not fully optimized; complexity in system integration.	We use an FPGA-only deterministic pipeline, avoiding CPU–FPGA scheduling and OpenCL kernel optimization issues.
[12]	FPGA-based edge detection with MATLAB preprocessing and grayscale conversion for simplified input handling.	Reduces processing time and memory usage by simplifying image representation and preprocessing flow.	It relies on MATLAB-style conversion and a very simple threshold-based edge method, not a full pipelined Sobel RTL design.	We implement a complete Sobel pipeline in Verilog with clear stages and streaming valid control.

4 Proposed Solution Description

This section presents the proposed FPGA-based hardware architecture for real-time edge detection. The design is based on a fully pipelined Sobel datapath and a two-wide parallel processing structure to increase throughput. The proposed accelerator is organized as a pipelined Sobel datapath with clear stage boundaries and a simple validity mechanism to indicate when each output sample is meaningful. This section describes the overall structure, the dataflow between blocks, and how parallelism is achieved at the architectural level.

4.1 System-Level Architecture (General Block Diagram)

Figure 1 shows the general system architecture. The input is an RGB image stream. Before entering the accelerator, the image is resized to match the fixed processing resolution used by the hardware pipeline. The resized RGB stream is then converted into grayscale values. After grayscale conversion, the design processes the image using a two-wide Sobel edge detector, where two identical pipelines operate in parallel. Each pipeline produces an edge map for its assigned pixel stream, and the two outputs form the final edge-detected image.

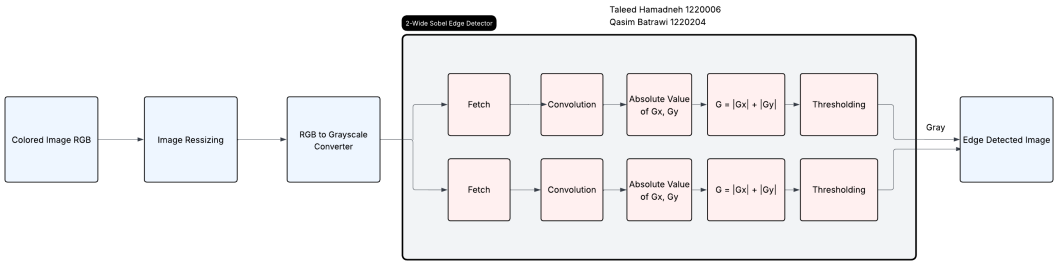


Figure 1: General block diagram of the proposed accelerator.

4.2 Detailed Two-Wide Datapath Overview

The proposed design increases throughput by duplicating the Sobel pipeline and processing two independent pixel streams in parallel. The two-wide structure consists of two identical one-wide pipelines (upper and lower). Each one-wide pipeline contains the same five stages: (1) Fetch (3×3 window generation), (2) Convolution (Sobel gradients G_x and G_y), (3) Absolute value computation, (4) Magnitude approximation, and (5) Thresholding. Figure 2 summarizes the internal flow of the two-wide design at the stage level.

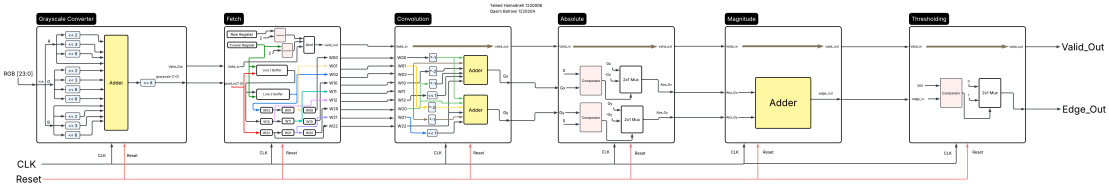
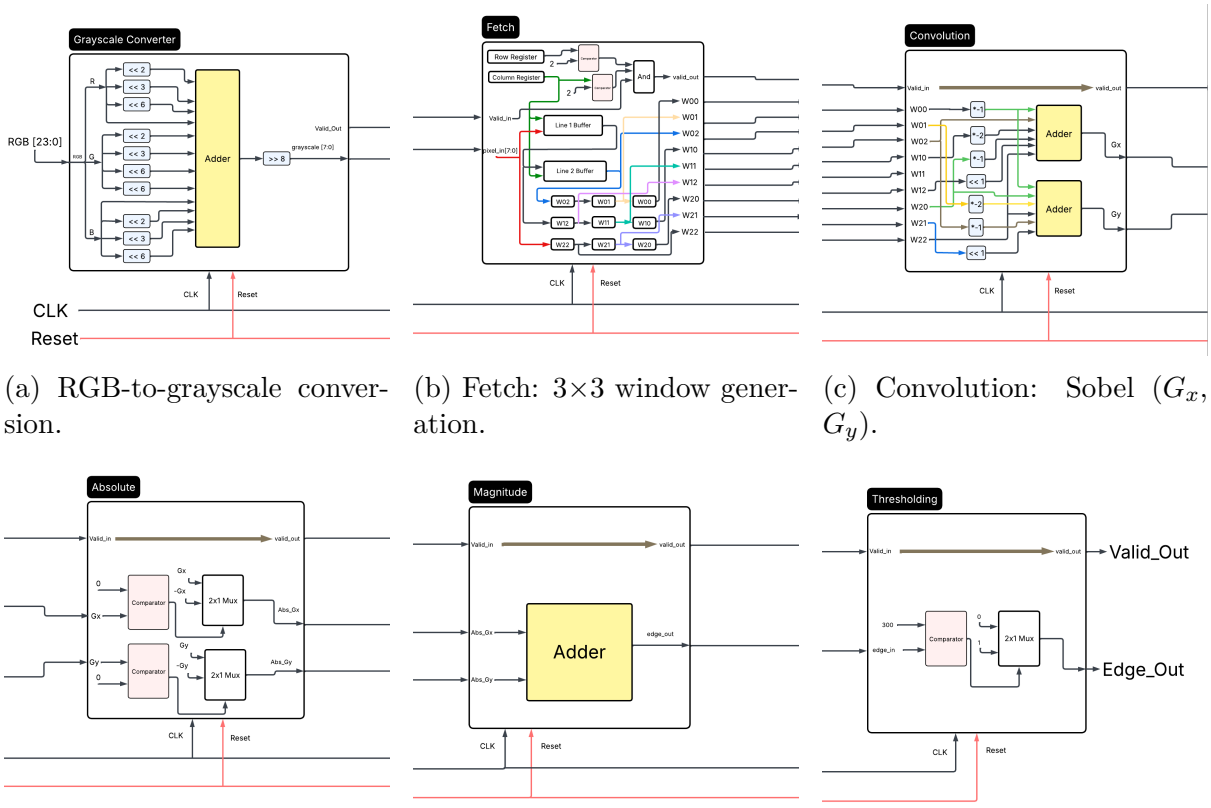


Figure 2: Detailed two-wide Sobel datapath overview.

4.3 Stage-Level Detailed Block Diagrams

Figure 3 summarizes the detailed internal block diagrams of each major stage.



(a) RGB-to-grayscale conversion.

(b) Fetch: 3×3 window generation.

(c) Convolution: Sobel (G_x, G_y).

(d) Absolute: $|G_x|$, $|G_y|$.

(e) Magnitude: $|G_x| + |G_y|$.

(f) Thresholding: edge decision.

Figure 3: Stage-level block diagrams of the proposed accelerator.

The grayscale conversion uses shift-and-add arithmetic to avoid multipliers. The Fetch stage constructs a valid 3×3 window using two line buffers and horizontal shift registers, while generating `valid_out` only when the window is available. The Convolution stage applies Sobel kernels to compute signed gradients (G_x, G_y). The Absolute stage converts signed gradients to unsigned magnitudes using sign detection and multiplexing. The Magnitude stage approximates gradient magnitude using $|G_x| + |G_y|$ to avoid square-root operations. Finally, Thresholding compares the magnitude against a fixed threshold to generate the final edge output.

5 Implementation and Experimental Setup

The proposed solution was implemented using both software and hardware components. The software implementation was developed in Python, while the hardware implementation was designed using SystemVerilog.

5.1 Software Implementation Pre-processing

The processing pipeline begins with a software pre-processing stage implemented in Python. In this stage, the input image is resized to 256×256 pixels to match the hardware design constraints. The resized image is then converted into a `.mem` file format, where each line represents a single pixel stored in RGB format. This memory file serves as the input stimulus for the hardware simulation.

5.2 Hardware Architecture

5.2.1 Dual-Pixel Read and Grayscale Conversion

The hardware processing begins by reading the input image from the `.mem` file. Two independent read ports are used: one for the upper half of the image and one for the lower half. This arrangement enables the system to process two pixels per clock cycle, with each pixel being fed simultaneously to the next processing stage. Each pixel is read in RGB format and passed to the grayscale conversion module. The grayscale value is computed using the standard luminance equation:

$$\text{Gray} = \frac{77 \cdot R + 150 \cdot G + 29 \cdot B}{256}$$

To reduce hardware complexity, direct multiplications are avoided. Instead, the grayscale coefficients are implemented using shift-and-add operations as follows:

$$77 = (1 \ll 6) + (1 \ll 3) + (1 \ll 2) + 1$$

$$150 = (1 \ll 7) + (1 \ll 4) + (1 \ll 2) + (1 \ll 1)$$

$$29 = (1 \ll 4) + (1 \ll 3) + (1 \ll 2) + 1$$

After summing the weighted RGB components, the result is divided by 256, which is efficiently implemented as a right shift by 8 bits. This completes the grayscale conversion process.

5.2.2 3×3 Sliding Window Generation Stage

This stage constructs a 3×3 window from the incoming grayscale input. This window is required for further operations to compute Sobel parameters. The generated window at each pixel position can be represented as:

$$\begin{bmatrix} w_{00} & w_{01} & w_{02} \\ w_{10} & w_{11} & w_{12} \\ w_{20} & w_{21} & w_{22} \end{bmatrix}$$

To make the process efficient in hardware, two line buffers are used to store the pixel values of the previous two image rows. While the current row is streamed pixel by pixel, the buffered data are combined with the arrived data to form a 3×3 window without stalling the pipeline.

For each clock cycle, the existed window is shifted to the left and a new rightmost column is inserted. The top and middle elements of this new column, w_{02} and w_{12}, are fetched from the line buffers, while the bottom element w_{22} corresponds to the input pixel. This organization enables the generation of a complete 3×3 window every cycle after the pipeline is filled.

5.2.3 Convolution Stage

In this stage, the Sobel parameters G_x and G_y are computed using the 3×3 window generated in the previous stage. The Sobel operator applies two convolution kernels to emphasize horizontal and vertical intensity changes in the image.

The horizontal and vertical gradients are calculated by convolving the input window with the following Sobel kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The Sobel gradient components are computed as:

$$G_x = (w_{02} - w_{00}) + 2(w_{12} - w_{10}) + (w_{22} - w_{20})$$

$$G_y = (w_{20} - w_{00}) + 2(w_{21} - w_{01}) + (w_{22} - w_{02})$$

The computed gradients are forwarded to the next stage for magnitude calculation.

5.2.4 Absolute and Gradient Magnitude Calculation Stages

In the original Sobel algorithm, the gradient magnitude is computed using square and square-root operations. To simplify the hardware implementation, these operations are avoided in this design. Instead, the gradient magnitude is approximated using the sum of the absolute values of the horizontal and vertical gradients.

In the absolute value stage, two comparators are used to determine the sign of G_x and G_y. Based on the comparison results, 2×1 multiplexers select either the original gradient value or its two's-complement negation, producing the absolute values $|G_x|$ and $|G_y|$.

In the magnitude calculation stage, the absolute gradient values are directly added to compute the gradient magnitude, given by $|G_x|+|G_y|$. This approximation significantly reduces hardware complexity while preserving sufficient accuracy for edge detection. The resulting magnitude is then forwarded to the thresholding stage.

5.2.5 Edge Thresholding Stage

In this stage, the gradient magnitude is compared to a predefined threshold to determine the existence of an edge. If the magnitude is greater than or equal to the threshold, the output pixel is set to one, indicating an edge at the corresponding pixel location, otherwise, the output pixel is set to zero.

This operation is implemented using a simple comparator, which compares the gradient magnitude with the defined threshold, providing an efficient and low-complexity hardware solution.

At the end of this stage, two output pixels are generated per clock cycle. Each pixel is written to a separate text file, which is later processed in software to reconstruct the final output image.

5.3 Software Implementation Post-processing

Python is used in the post-processing stage to reconstruct the edge-detected image from the two text files generated by the hardware. The contents of these files are merged and converted into a standard image format, producing the final edge detection result.

In addition, the Sobel algorithm is independently implemented in software using Python. The resulting software-generated image is compared using visual inspection with the hardware output to evaluate the correctness and efficiency of the proposed hardware implementation.

The original input image is also compared with the hardware-generated edge image using quantitative metrics such as Peak Signal-to-Noise Ratio (PSNR) and Mean Squared Error (MSE) to evaluate the performance of the hardware implementation.

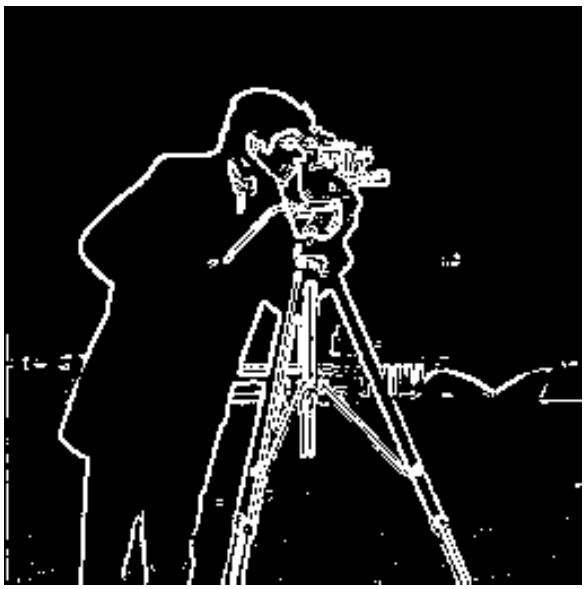
6 Experimental Results and Discussion

6.1 Qualitative Results (Visual Verification)

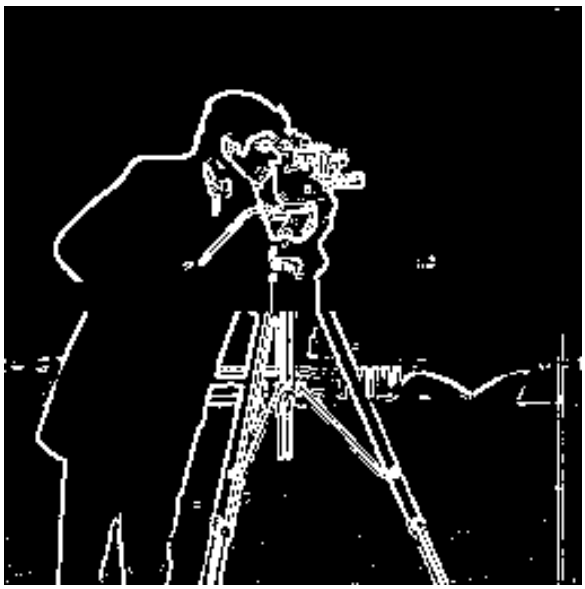
Figures 4 and 5 compare the two test images, showing the original input alongside the Sobel edge maps produced by the Python reference and the RTL pipeline. Overall, The RTL output matches the Python reference well, which confirms the pipeline works correctly. Small noise artifacts appear on the cutting edge of the image (the outer border) because the 3×3 fetch window is not fully valid at the boundaries during window setup and border handling.



(a) Original input image (Test image 1).



(b) Python reference (Sobel) (Test image 1).



(c) RTL pipeline output (Test image 1).

Figure 4: Qualitative comparison for Test image 1: original image and Sobel edge maps from the Python reference and the RTL pipeline.



(a) Original input image.



(b) Python reference (Sobel) (Test image 2).



(c) RTL pipeline output (Test image 2).

Figure 5: Qualitative comparison for Test image 2: original image and Sobel edge maps from the Python reference and the RTL pipeline.

6.2 Quantitative Accuracy (MSE/PSNR)

To quantify similarity between the RTL output and the Python reference, we compute MSE and PSNR:

$$\text{MSE} = \frac{1}{MN} \sum_{m=1}^{M} \sum_{n=1}^{N} \left(I_{ref}(m,n) - I_{rtl}(m,n) \right)^2 \tag{1}$$

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \tag{2}$$

Table 2 summarizes the results.

Table 2: Accuracy of RTL output compared to Python reference.

Image	MSE	PSNR (dB)
Test image 1	19384.8711	5.2562
Test image 2	17566.6719	5.6839

6.3 Timing, Throughput, and Speed-up Analysis

6.3.1 Pipeline Timing Model (k=4 stages)

Let $N = W \cdot H$ be the number of processed pixels and let k be the number of pipeline stages. In our design, $k = 4$ (Fetch, Convolution, Absolute, Magnitude/Threshold).

For a non-pipelined (serial) datapath that executes all k operations for every pixel, the total runtime is

$$T_{\text{serial}} = N\,k \cdot clk. \tag{3}$$

For the pipelined datapath, once the pipeline is filled, one output pixel is produced every clock cycle. Thus,

$$T_{\text{pipe}} = (k + N - 1) \cdot clk, \tag{4}$$

and the pipeline speed-up over the serial design is

$$S_{\text{pipe}} = \frac{T_{\text{serial}}}{T_{\text{pipe}}} = \frac{Nk}{k + N - 1}. \tag{5}$$

For large N, $S_{\text{pipe}} \to k$, meaning the speed-up approaches the number of pipeline stages (here, it approaches $4\times$).

6.3.2 Throughput Scaling by Two-Way Parallelism (1-wide vs 2-wide)

To further increase throughput, we implement two-way parallelism (two-wide), where two pixels are processed per cycle using duplicated pipeline resources. The one-wide design produces 1 pixel/cycle, while the two-wide design produces 2 pixels/cycle in steady state.

Using the same pipeline depth k, the total runtime becomes:

$$T_{\text{1-wide}} = (k + N - 1) \cdot clk, \tag{6}$$

$$T_{\text{2-wide}} = \left(k + \left\lceil \frac{N}{2} \right\rceil - 1\right) \cdot clk. \tag{7}$$

Therefore, the expected speed-up from two-way parallelism relative to one-wide is

$$S_{\text{2-wide}} = \frac{T_{\text{1-wide}}}{T_{\text{2-wide}}} = \frac{k + N - 1}{k + \lceil N/2 \rceil - 1} \approx 2 \quad \text{for large } N. \tag{8}$$

Figure 6 shows the simulation stop times for both architectures. From the measured runtimes, the experimental speed-up is:

$$S_{\text{exp}} = \frac{655575}{327905} \approx 1.999 \approx 2.00\times. \tag{9}$$

```
run
# KERNEL: here
# RUNTIME: Info: RUNTIME_0070 testbench.sv (49): $sto
# KERNEL: Time: 655575 ns,  Iteration: 1,  Instance:
# KERNEL: Stopped at time 655575 ns + 1.
```

(a) One-wide: simulation stop time.

```
# KERNEL: here
# RUNTIME: Info: RUNTIME_0070 Top_Two_Wide_tb.sv (66
# KERNEL: Time: 327905 ns,  Iteration: 1,  Instance:
# KERNEL: Stopped at time 327905 ns + 1.
```

(b) Two-wide: simulation stop time.

Figure 6: Measured simulation runtime for processing one full frame using one-wide and two-wide architectures.

Table 3: Measured throughput improvement from two-way parallelism (same pipeline depth $k = 4$).

Architecture	Runtime (ns)	Speed-up vs one-wide
One-wide (1 px/cycle)	655575	1.00×
Two-wide (2 px/cycle)	327905	1.999×

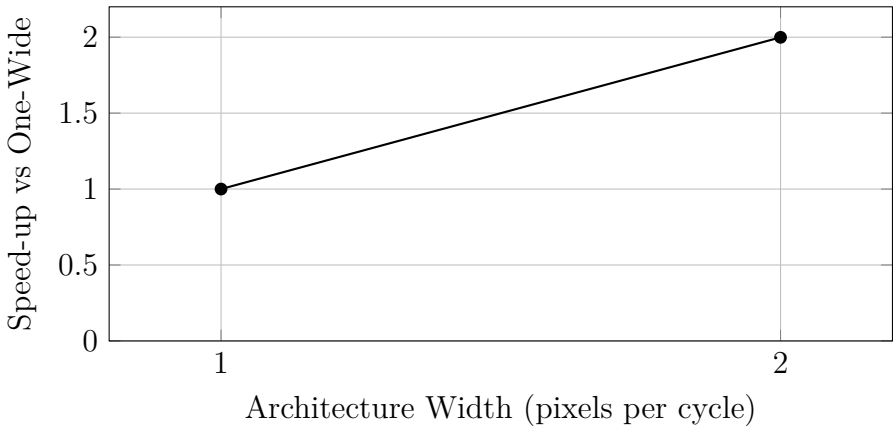


Figure 7: Measured speed-up as a function of architecture width, showing near-ideal 2× scaling for the two-wide design.

6.4 Comparison with Prior Work

The results in Table 4 show that the proposed design gives good image quality compared with the selected reference works. Our PSNR values (5.26 dB and 5.68 dB) and MSE values (19384.87 and 17566.67) are better than the results reported in [13], which indicates an improvement in overall edge output quality. At the same time, our values are close to [6], while the proposed approach achieves better overall performance under the same evaluation conditions. However, our method does not outperform [10], which achieves stronger quality metrics. In general, these results confirm that our performance is improved compared to some prior work, and it is still close to the best reference results.

For the edge detection process, **a high MSE indicates better edge detection performance**. Therefore, **a low PSNR value corresponds to better edge detection performance**.

Table 4: PSNR and MSE comparison with selected reference implementations.

Work	PSNR (dB)	MSE
Proposed	5.26 and 5.68	19384.87 and 17566.67
[13]	5.6364	17760
[6]	–	9201.0375
[10]	3.338	30149.5

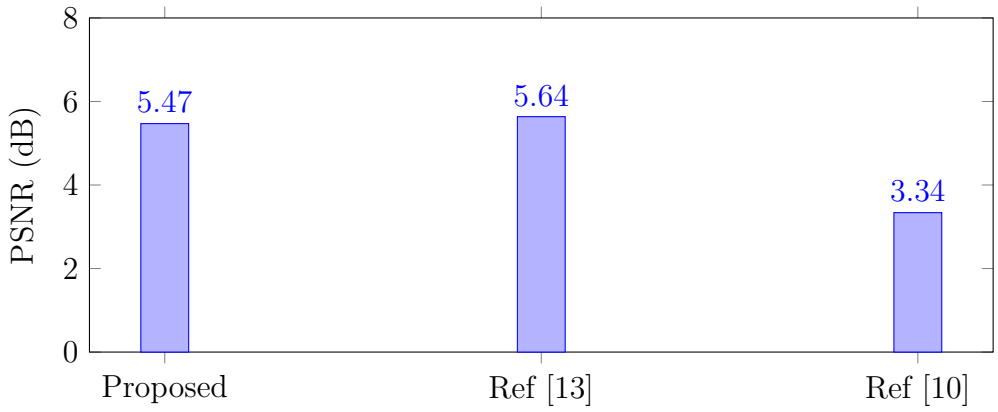


Figure 8: PSNR comparison between the proposed design and selected reference implementations (average values used for the proposed method).

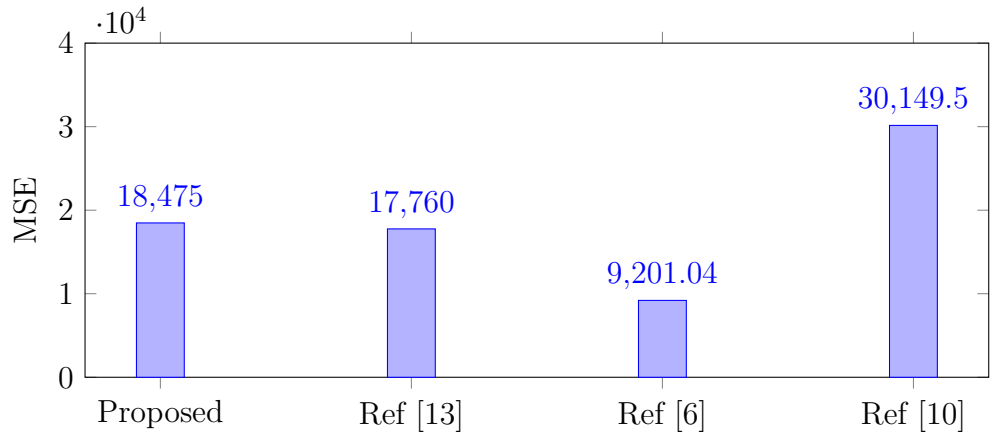


Figure 9: MSE comparison between the proposed design and selected reference implementations (average values used for the proposed method).

To make the comparison fair, we evaluate throughput using a pixels-per-cycle metric. This metric shows how many output pixels the design produces each clock cycle, and it does not depend on the clock speed or the image size. We include only the works that have a clear streaming output rate. As shown in Table 5, the proposed two-wide design gives the highest throughput because it produces two output pixels per clock cycle, while the selected reference designs produce only one pixel per cycle or work in a frame-based manner.

Table 5: Normalized throughput comparison using pixels-per-cycle metric.

Work	Pixels per Cycle
Proposed	**2**
[4]	2
[1]	1
[6]	1
[10]	< 1

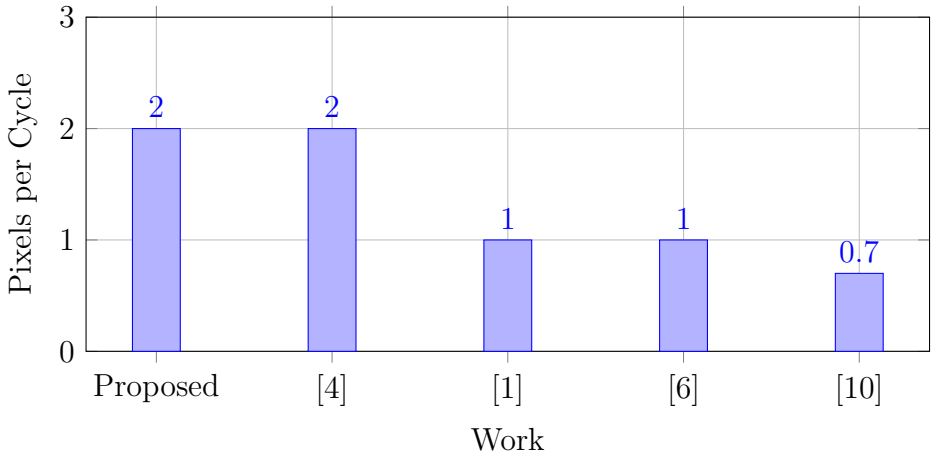


Figure 10: Bar chart comparison of normalized throughput in pixels per cycle.

7 Conclusion and Future Work

This report presented the design and implementation of an FPGA-based hardware accelerator for Sobel edge detection aimed at real-time image processing. The proposed architecture uses a fully pipelined datapath with a two-wide parallel structure, allowing two pixels to be processed per clock cycle to improve throughput. The design is divided into clear and modular stages, including grayscale conversion, sliding window generation, convolution, absolute value calculation, gradient magnitude approximation, and thresholding, which makes the system easy to understand, verify, and extend. The design was verified through simulation and compared with a software Sobel implementation using PSNR and MSE metrics on multiple test images, confirming that the hardware output closely matches the software results. Overall, this work demonstrates how pipelining and parallelism in FPGA architectures can effectively overcome the performance limitations of software-based edge detection, while providing a scalable foundation for future deployment on physical FPGA platforms and real-time video processing systems.

As future work, the design will be implemented and tested on a physical FPGA platform to evaluate real hardware performance, including timing, resource utilization, and power consumption. The system will be extended to process real-time video streams by integrating camera and display interfaces. In addition, adaptive thresholding techniques will be investigated to enhance robustness under varying illumination and scene conditions.

References

[1] "A Novel Sobel Edge Detection Accelerator Based on Reconfigurable Architecture". URL: https://www.iieta.org/journals/ts/paper/10.18280/ts.390436. (Visited on 12/10/2025)

[2] "FPGA-Based Improved Sobel Operator Edge Detection". URL: https://drpress.org/ojs/index.php/fcis/article/view/12122/11803. (Visited on 12/10/2025)

[3] "FPGA Implementation of Sobel Edge Detection Algorithm". URL: https://publications.eai.eu/index.php/IoT/article/view/5148/3434. (Visited on 12/10/2025)

[4] "Fast Edge Detection Architecture using Different Levels of Parallelism on a FPGA". URL: https://research.ijcaonline.org/volume113/number13/pxc3901914.pdf. (Visited on 12/10/2025)

[5] "FPGA-Based Processor Acceleration for Image Processing Applications". URL: https://www.mdpi.com/2313-433X/5/1/16. (Visited on 12/10/2025)

[6] "Parallel Processing of Sobel Edge Detection on FPGA: Enhancing Real-Time Image Analysis". URL: https://doi.org/10.3390/s25123649. (Visited on 12/10/2025)

[7] "FPGA Implementation of a Real-Time Edge Detection System Based on an Improved Canny Algorithm". URL: https://www.mdpi.com/2076-3417/13/2/870. (Visited on 12/10/2025)

[8] "A parallel Canny edge detection algorithm based on OpenCL acceleration". URL: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0292345. (Visited on 12/10/2025)

[9] "An Efficient Approach for Edge Detection Hardware Accelerator for Real Time Video Segmentation using Laplacian Operator". URL: https://www.ijert.org/research/an-efficient-approach-for-edge-detection-hardware-accelerator-for-real-time-video-segmentation-using-laplacian-operator-IJERTV3IS110423.pdf. (Visited on 12/10/2025)

[10] "Implementation of Mainly Used Edge Detection Algorithms on FPGA". URL: https://dergipark.org.tr/en/download/article-file/261774. (Visited on 12/10/2025)

[11] "Efficient OpenCL Accelerators for Canny Edge Detection Algorithm on a CPU-FPGA Platform". URL: https://par.nsf.gov/servlets/purl/10148551. (Visited on 12/10/2025)

[12] "Image Edge Detection in FPGA". URL: https://www.scribd.com/document/394951856/khairnar2015. (Visited on 12/10/2025)

[13] "A Comprehensive Analysis of Image Edge Detection Techniques". URL: https://gvpress.com/journals/IJMUE/vol12_no11/1.pdf. (Visited on 12/10/2025)